

Course Project Proposal



Parallel Odd-Even Sort Simulation

Course: Analysis of Algorithm

Group Members

Name	Roll Number
Mishaal Aqdas	NUM-BSCS-2024-35
Najia Nayab	NUM-BSCS-2024-62
Shahbaz Ali	NUM-BSCS-2024-73

Instructor: Muhammad Shahzad Arif

Date: 17 May 2026

NAMAL UNIVERSITY MIANWALI

Department of Computer Sciences

1. Project Overview

1.1 Background

With the increasing demand for high-performance computing, parallel computing has become an important solution for processing large-scale data efficiently. Many modern systems rely on parallel algorithms to improve speed and performance by executing multiple operations simultaneously.

Sorting algorithms are widely used in databases, networking, operating systems, and data processing applications. Among these algorithms, Odd-Even Transposition Sort is well-suited for parallel execution because it performs comparisons in separate Odd and Even phases that can run concurrently. This makes it a useful algorithm for understanding synchronization and parallel processing concepts.

1.2 Problem Statement

Although Odd-Even Sort is commonly discussed in theory, students often struggle to visualize how parallel comparison and swapping occur during execution. Most existing examples are static and do not clearly demonstrate concurrent processing behavior.

This project aims to develop a graphical simulation of Parallel Odd-Even Transposition Sort to help users better understand the working mechanism of parallel sorting algorithms through real-time visualization.

2. Proposed Scope & Objectives

2.1 Project Scope

In Scope

- Implementation of Parallel Odd-Even Transposition Sort.
- Graphical visualization of Odd and Even execution phases.
- Configurable array size and execution speed.
- Performance comparison with sequential Bubble Sort.
- Analysis of execution time, comparisons, and swaps.

Out of Scope

- GPU-based implementation.
- Distributed hardware deployment.
- Advanced distributed sorting techniques.

2.2 Objectives

1. To implement the Odd-Even Transposition Sorting algorithm in C++.
2. To simulate parallel execution using multithreading concepts.
3. To visualize comparison and swap operations during Odd and Even phases.
4. To analyze the algorithm in terms of:
 - Time Complexity
 - Execution Time
 - Number of Comparisons
 - Number of Swaps
5. To compare Parallel Odd-Even Sort with traditional sorting algorithms such as Bubble Sort, Selection Sort, and Insertion Sort.
6. To develop a simple and user-friendly educational simulation tool.

3. Methodology & Tools

3.1 Methodology

1. Research & Design

Study the Odd-Even Transposition Sort algorithm and parallel computing concepts. Prepare the overall design and execution flow for the simulation.

2. Algorithm Implementation

Implement both sequential and parallel versions of Odd-Even Sort in C++. Parallel behavior will be simulated using multithreading concepts.

3. Visualization Development

Develop a graphical interface to display array elements, active comparisons, swaps, and phase transitions in real time.

4. Testing & Analysis

Test the algorithm with different input sizes and analyze performance using execution time and comparison statistics.

5. Documentation & Presentation

Prepare the final report and presentation, including implementation details, observations, and results.

3.2 Tools & Technologies

- **Programming Language:** C++ (Primary), Python (optional)
- **IDE:** Code::Blocks / Visual Studio Code
- **Visualization Library:** SFML, Python Visualization Libraries
- **Concepts Used:** Multithreading, Synchronization, Parallel Processing

4. Expected Outcomes

- A working simulation of Parallel Odd-Even Sort.
- Real-time visualization of sorting phases.
- Comparative analysis between sequential and parallel sorting methods.
- Better understanding of parallel algorithm execution and synchronization concepts.

5. Conclusion

This project will provide a practical demonstration of parallel sorting techniques through visualization and performance analysis. By combining implementation and graphical simulation, the project will help students better understand the behavior of parallel algorithms and their advantages over sequential approaches.